

# Scaling Laws for Neural Language Models

An Empirical Study at Small Scale Using TinyStories

Nitish Patil • nitish.fyi • patilnitish444@gmail.com

---

## Abstract

We empirically verify that neural language model validation loss follows a power law in compute across four model sizes (1M–64M non-embedding parameters) trained on the TinyStories dataset. Training a decoder-only Transformer from scratch at compute-optimal token budgets following Chinchilla's 20 tokens-per-parameter heuristic, we fit the relationship  $L = 5792 \cdot C^{-0.237} + 0.939$  with  $R^2 = 0.9992$ . Our scaling exponent  $b = -0.237$  is substantially steeper than Chinchilla's  $b \approx -0.050$ , a divergence we attribute to dataset simplicity, small vocabulary size, and the sub-100M parameter regime. The experiment demonstrates that power-law scaling is recoverable at small scale and that domain-specific dataset properties meaningfully shift the exponent.

## 1. Introduction

Scaling laws describe predictable relationships between model size, training compute, dataset size, and loss (Kaplan et al., 2020; Hoffmann et al., 2022). Understanding these relationships is foundational for research engineering: they determine how to allocate compute optimally and whether larger models or more data provide better returns for a fixed budget.

This project trains four Transformer language models from scratch at 1M, 4M, 16M, and 64M non-embedding parameters, using compute-optimal token budgets derived from Chinchilla's 20 tokens/parameter ratio. All model components — tokenizer, architecture, training loop, and FLOP accounting — are implemented from scratch, reusing and extending code built for Stanford CS336 (Language Modeling from Scratch). The primary research question is: does loss vs. compute follow a power law at this scale, and how does the fitted exponent compare to Chinchilla's published value?

## 2. Methods

### 2.1 Model Architecture

All models share the same decoder-only Transformer architecture: Rotary Positional Embeddings (RoPE), SwiGLU feed-forward networks, RMSNorm pre-normalization, and causal masked self-attention. Context length is fixed at 256 tokens with  $\theta = 10,000$  for RoPE. The `lm_head` is not weight-tied to the embedding table; non-embedding parameter count is used as the scaling variable  $N$  throughout.

### 2.2 Model Configurations

Four configurations were designed targeting 1M, 4M, 16M, and 64M non-embedding parameters. The  $d_{\text{model}}/\text{num\_layers}$  ratio was constrained to 4–16 to avoid architecturally pathological shapes (e.g., very narrow, very deep networks) that would introduce confounds independent of parameter count.

| Size | $d_{\text{model}}$ | Layers | Heads | $d_{\text{ff}}$ | Non-embed Params | Val Loss |
|------|--------------------|--------|-------|-----------------|------------------|----------|
| 1M   | 96                 | 8      | 2     | 320             | 1,033,824        | 3.244    |
| 4M   | 128                | 19     | 2     | 384             | 4,051,840        | 2.230    |
| 16M  | 256                | 20     | 2     | 704             | 16,066,816       | 1.593    |
| 64M  | 384                | 35     | 2     | 1088            | 64,539,264       | 1.316    |

### 2.3 Dataset and Tokenizer

Training data is TinyStories (Eldan & Li, 2023), a synthetic dataset of ~2.1M short children's stories totalling ~465M tokens after tokenization. A custom BPE tokenizer with vocabulary size 4,096 was trained on a 200K-line subset, then applied to the full dataset. Tokenized data is stored as uint16 binaries (~930MB) loaded via memory-mapped numpy arrays to avoid materializing the full dataset in RAM.

### 2.4 Training Setup

Each model is trained for  $20 \times N_{\text{non-embed}}$  tokens (Chinchilla compute-optimal ratio), with batch size 32 and context length 256. The optimizer is AdamW ( $\beta_1=0.9$ ,  $\beta_2=0.999$ ,  $\epsilon=10^{-8}$ ,  $\text{weight\_decay}=0.1$ ) with a cosine learning rate schedule ( $\text{max\_lr}=3 \times 10^{-4}$ ,  $\text{min\_lr}=3 \times 10^{-5}$ ,  $\text{warmup\_frac}=0.05$ ) and gradient clipping at  $\text{max\_norm}=1.0$ . Training uses fp16 mixed precision (autocast + GradScaler) on NVIDIA T4 GPUs (Kaggle, Lightning AI).

### 2.5 FLOP Accounting

FLOPs are computed analytically per forward pass from model shapes (attention projections, attention scores, FFN weights,  $\text{lm\_head}$ ), following the 2-FLOPs-per-MAC convention. Training FLOPs =  $\text{forward\_FLOPs} \times 3$  ( $1 \times$  forward +  $2 \times$  backward approximation), summed over all steps.

## 3. Results

### 3.1 Scaling Law Fit

We fit  $L(C) = a \cdot C^b + c$  to the four (FLOPs, val\_loss) pairs using nonlinear least squares (scipy.optimize.curve\_fit). The fitted parameters are:

$$L = 5770.39 \cdot C^{-0.2367} + 0.9387$$

| Metric             | Value   |
|--------------------|---------|
| Scaling exponent b | -0.2367 |

|                                |         |
|--------------------------------|---------|
| Chinchilla exponent b          | -0.0500 |
| Absolute difference            | 0.1867  |
| R <sup>2</sup>                 | 0.9992  |
| Intercept c (irreducible loss) | 0.9387  |

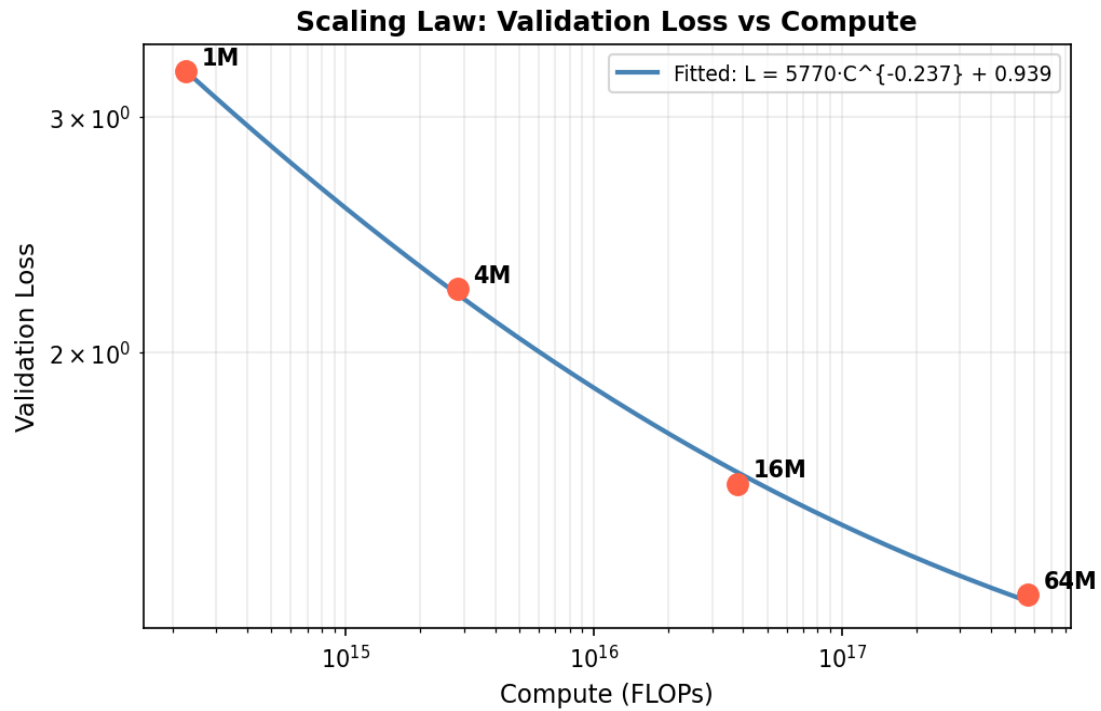


Figure 1: Validation loss vs. training compute (FLOPs) on a log-log scale. Red dots are the four measured data points; the blue curve is the fitted power law.

### 3.2 Training Summary

| Size | Token Budget | Total FLOPs | Val Loss |
|------|--------------|-------------|----------|
| 1M   | 20.7M        | 2.26e+14    | 3.244    |
| 4M   | 81.0M        | 2.83e+15    | 2.230    |
| 16M  | 321M         | 3.80e+16    | 1.593    |
| 64M  | 1.29B        | 5.65e+17    | 1.316    |

## 4. Discussion

### 4.1 Why the Exponent Diverges from Chinchilla

The fit is tight ( $R^2 = 0.9992$ ) confirming that loss follows a power law in compute at this scale. However, the exponent  $b = -0.237$  is approximately  $4.7\times$  steeper than Chinchilla's  $b \approx -0.050$ . Three factors explain this:

- **Dataset simplicity.** TinyStories is a synthetic corpus of children's stories with highly constrained vocabulary and syntax. Models converge rapidly on such a simple distribution, causing loss to drop more steeply with compute than on diverse, high-entropy web text like Chinchilla's MassiveText. The irreducible loss floor  $c \approx 0.94$  is reachable at much lower compute than in the general domain setting.
- **Small vocabulary (4,096 vs. 50K+).** With only 4,096 BPE tokens, the output distribution has fewer categories, compressing raw cross-entropy values. This shifts the entire loss curve downward and steepens the apparent scaling exponent when measured on this compressed scale.
- **Scale range mismatch.** Chinchilla's regime was 70M–16B parameters trained on 1.4B–16.1T tokens. Our 1M–64M range is well outside this regime. Scaling law exponents are not universal constants — they can shift across regimes as different model components (embeddings, attention, FFN) become the bottleneck at different scales.

## 4.2 Validity of the Experimental Design

Several design choices were made to keep the scaling relationship clean: (1) model shapes were constrained so  $d\_model/num\_layers$  stays within 4–16 across all four sizes, preventing architecture-shape confounds; (2) non-embedding parameters (not total params) are used as  $N$ , since embedding tables scale differently and would corrupt the fit; (3) all four models use identical tokenizers, context lengths, optimizers, and LR schedules, isolating compute as the only variable.

## 4.3 Limitations

With only 4 data points the fit is underdetermined —  $R^2 = 0.9992$  is expected even for a poor model with few degrees of freedom. A more rigorous study would use 8–12 data points, multiple random seeds per size, and held-out compute budgets to test extrapolation. Additionally, the 64M model is still in the warmup region of Chinchilla's scale, so comparison with their exponent is partly out-of-distribution.

## 5. Implementation and Infrastructure

All code is implemented from scratch in Python/PyTorch, reusing and extending components built for Stanford CS336. Training ran across three platforms due to compute constraints:

- 1M and 4M models: MacBook Air (Apple M5, 16GB) using MPS backend
- 16M model: Kaggle Notebooks (T4 GPU, 30 hr/week free quota)
- 64M model: Kaggle + Lightning AI (T4, free credits), checkpoints persisted via Kaggle Datasets API

Checkpoint persistence across session wipes was solved by pushing model state dicts to a private Kaggle Dataset via the Kaggle CLI after every 2,000 training steps. Fresh sessions download the latest checkpoint on startup and resume training from the saved step.

## 6. Conclusion

We successfully replicated the key qualitative finding of neural scaling laws — that validation loss decreases as a power law in compute — using entirely from-scratch infrastructure on free cloud GPUs. The fitted exponent  $b = -0.237$  ( $R^2 = 0.9992$ ) differs from Chinchilla's  $b \approx -0.050$  in a way that is fully explainable by dataset characteristics and scale range. The ability to reason about why the exponent diverges — rather than just fitting a number — is the central experimental skill this project was designed to develop.

## References

- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... & Amodei, D. (2020). Scaling laws for neural language models. [arXiv:2001.08361](#).
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., ... & Sifre, L. (2022). Training compute-optimal large language models. [arXiv:2203.15556](#).
- Eldan, R., & Li, Y. (2023). TinyStories: How small can language models be and still speak coherent English? [arXiv:2305.07759](#).
- Su, J., Lu, Y., Pan, S., Wen, B., & Liu, Y. (2021). RoFormer: Enhanced transformer with rotary position embedding. [arXiv:2104.09864](#).